

Proyecto Funcional

Asignación de Horarios Universitarios con GNU Prolog

Eddy Ramírez Jiménez

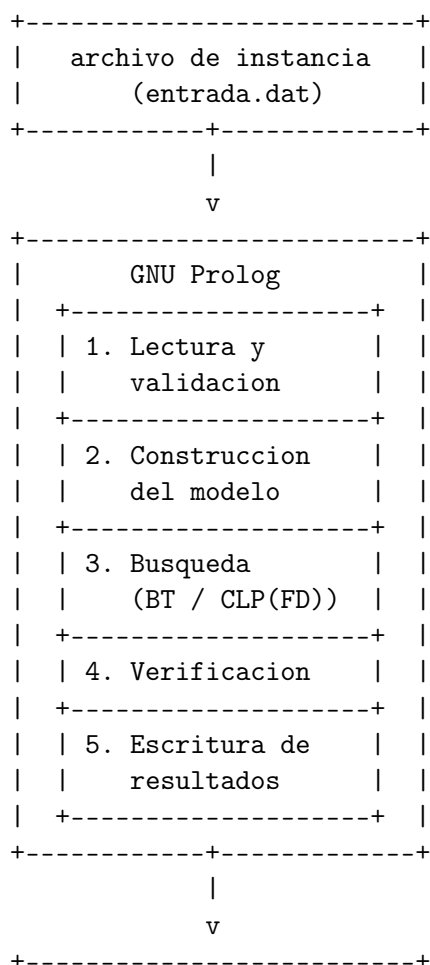
16 de septiembre de 2026

1. Descripción general

El objetivo del proyecto es desarrollar un sistema, implementado íntegramente en **GNU Prolog** y ejecutado sobre **Linux**, que resuelva el problema de **asignación de horarios universitarios** (*university timetabling*).

El sistema deberá recibir la descripción completa de una instancia (cursos, grupos, profesores, aulas, franjas horarias y restricciones) y determinar una asignación de cada sesión de clase a un aula, un día y una franja horaria. La asignación producida deberá satisfacer **todas las restricciones duras** y minimizar, en la medida de lo posible, el costo asociado a las **restricciones blandas**.

La arquitectura general del sistema será:



```
|    archivo de horario    |
|    (salida.txt)         |
+-----+
```

El sistema será invocado desde la línea de comandos mediante un guion de envoltura que invoca a GNU Prolog con la instancia y los parámetros de ejecución seleccionados.

2. Objetivos

Al finalizar el proyecto, el estudiante deberá demostrar dominio de los siguientes conceptos:

- Programación lógica declarativa mediante Prolog.
- Representación de un problema real mediante hechos y reglas.
- Uso de listas y recursión para el procesamiento de estructuras de datos.
- Definición de predicados con múltiples modos de uso.
- Manejo de entrada y salida de archivos en GNU Prolog.
- Resolución de problemas mediante *backtracking*.
- Programación con restricciones sobre dominios finitos (CLP(FD)).
- Definición y clasificación de restricciones duras y blandas.
- Uso de heurísticas de selección de variable y de valor.
- Técnicas de poda (*forward checking*, propagación).
- Estrategias de optimización mediante *branch and bound*.
- Medición y análisis del rendimiento de un programa lógico.
- Diagnóstico de instancias inválidas o insatisfacibles.

3. Especificación del sistema

El sistema recibirá como entrada un archivo de texto con la descripción de la instancia, el nombre del archivo de salida y, opcionalmente, la estrategia de resolución y los límites de búsqueda.

La ejecución deberá seguir, conceptualmente, las siguientes etapas:

1. El programa recibe el nombre del archivo de instancia.
2. El programa abre y lee el archivo de instancia.
3. El programa valida la sintaxis de cada línea y la coherencia de los datos (referencias existentes, rangos válidos, duplicados).
4. El programa construye la base de hechos que representa la instancia.
5. El programa calcula el tamaño del espacio de búsqueda y el número de sesiones a programar.
6. El programa construye las variables de decisión y sus dominios.
7. El programa impone las restricciones duras.

8. El programa ejecuta la búsqueda según la estrategia seleccionada.
9. El programa recolecta la solución y calcula el costo de las restricciones blandas.
10. El programa verifica de forma independiente que la solución cumple todas las restricciones duras.
11. El programa escribe el archivo de salida con el horario, el detalle de las asignaciones y las métricas de ejecución.

Si la instancia no admite solución, el programa deberá informarlo explícitamente y reportar los datos que permitan diagnosticar la causa.

4. Modelo del problema

4.1. Dominio

El dominio del problema está formado por los siguientes conjuntos:

- D : conjunto de días lectivos, identificados por un carácter (por ejemplo L, M, X, J, V).
- F : conjunto de franjas horarias por día, numeradas $1, 2, \dots, |F|$.
- P : conjunto de profesores.
- A : conjunto de aulas, cada una con una capacidad y un tipo.
- C : conjunto de pares (curso, grupo).

La franja f del día d abarca el intervalo de tiempo:

$$[t_0 + (f - 1) \cdot \Delta, t_0 + f \cdot \Delta)$$

donde t_0 es la hora de inicio configurada y Δ la duración de la franja en minutos.

4.2. Variables de decisión y dominios

Para cada par curso-grupo $c \in C$ con s_c sesiones semanales se crean s_c variables de decisión. Cada variable $X_{c,i}$ representa la i -ésima sesión de ese grupo y toma valores en:

$$X_{c,i} \in A \times D \times \{1, \dots, |F| - d_c + 1\}$$

donde d_c es la duración en franjas consecutivas de cada sesión del grupo c . El valor concreto es una terna:

$$X_{c,i} = \langle a, d, f \rangle$$

que significa que la i -ésima sesión de c se imparte en el aula a , el día d , ocupando las franjas $f, f + 1, \dots, f + d_c - 1$.

El espacio de búsqueda del modelo más ingenuo es:

$$\prod_{c \in C} (|A| \cdot |D| \cdot (|F| - d_c + 1))^{s_c}$$

y crece de forma exponencial con el número de sesiones, razón por la cual el proyecto exige estrategias de poda y de ordenamiento.

4.3. Restricciones duras

Las siguientes restricciones son obligatorias y no pueden violarse bajo ninguna circunstancia. **Implícitas.** Deben cumplirse siempre, sin necesidad de declararlas en el archivo de instancia:

1. **No solapamiento de grupo.** Dos sesiones del mismo par curso-grupo no pueden ocupar la misma franja del mismo día.
2. **No solapamiento de profesor.** Un profesor no puede tener dos sesiones simultáneas.
3. **No solapamiento de aula.** Un aula no puede alojar dos sesiones simultáneas.
4. **Capacidad.** La capacidad del aula asignada debe ser mayor o igual al número de inscritos del grupo.
5. **Tipo de aula.** Una sesión que requiere laboratorio sólo puede asignarse a un aula de tipo laboratorio o mixta; una sesión de tipo **teoría** no puede asignarse a un aula de tipo laboratorio.
6. **Disponibilidad del profesor.** Cada sesión debe comenzar en una franja en la que el profesor esté disponible.
7. **Máximo de franjas del profesor.** La suma de las franjas ocupadas por todas las sesiones de un profesor no puede exceder su límite.

Declaradas. Se expresan explícitamente en el archivo de instancia: **aula_fija**, **prohibida** y **excluyentes** (véase la sección **Formato de entrada**).

4.4. Restricciones blandas

Las restricciones blandas pueden violarse, pero cada violación agrega un costo. El objetivo del sistema es encontrar una solución factible de **costo mínimo**.

- **prefiere:** penalización de w si el grupo no tiene ninguna sesión que comience en la franja indicada.
- **compacta:** penalización de w por cada hueco del grupo.
- **Huecos de profesor** (implícita, opcional): penalización por cada franja libre entre dos franjas ocupadas por el mismo profesor en un mismo día.
- **Huecos de grupo** (implícita, opcional): penalización por cada franja libre entre dos franjas ocupadas por el mismo grupo en un mismo día.

El costo total de una solución S se define como:

$$\text{costo}(S) = \sum_k w_k \cdot (\text{violaciones de } r_k \text{ en } S)$$

Se entiende por **hueco** una franja libre situada estrictamente entre la primera y la última franja ocupada por el grupo o el profesor en un mismo día.

5. Formato de entrada

La instancia se describe en un archivo de texto plano, codificado en UTF-8, con extensión **.dat**. El programa deberá leer y validar este archivo sin depender de ninguna utilidad externa.

5.1. Estructura general

El archivo está compuesto por líneas. Cada línea es un comentario, un encabezado de sección, un registro de datos o una línea en blanco. La gramática es la siguiente:

```
<archivo> ::= { <linea> }
<linea> ::= <blanco> | <comentario> | <seccion> | <registro>
<blanco> ::= <fin>
<comentario> ::= "#" { <caracter> } <fin>
<seccion> ::= "#:" <nombre> <fin>
<registro> ::= <campo> { ";" <campo> } <fin>
<campo> ::= { <caracter> - { ";", "#", <fin> } }
<fin> ::= "\n" | <eof>
<nombre> ::= "CONFIG" | "AULA" | "PROFESOR" | "CURSO"
           | "IMPARTE" | "RESTRICCION"
```

Reglas adicionales:

- Las líneas en blanco y los comentarios (que comienzan con # seguido de cualquier carácter distinto de :) se ignoran.
- Los encabezados de sección comienzan con #: y deben aparecer a lo sumo una vez por sección.
- El orden de las secciones es indistinto.
- Los espacios al inicio y al final de cada campo se eliminan.
- Un campo puede estar vacío cuando la sección lo permita.
- Una sección puede contener cero o más registros.
- Todo campo numérico debe ser un entero decimal no negativo.

5.2. Sección CONFIG

Registros de la forma `clave;valor`. Todas las claves son obligatorias y deben aparecer exactamente una vez.

Clave	Descripción	Ejemplo
dias	Códigos de día, separados por coma, un carácter cada uno	L,M,X,J,V
franjas	Cantidad de franjas horarias por día	7
inicio	Hora de inicio de la primera franja, formato HH:MM	07:00
duracion_franja	Duración de cada franja, en minutos	60

5.3. Sección AULA

Registros de la forma `codigo;capacidad;tipo`.

Campo	Descripción	Ejemplo
codigo	Identificador único del aula	LAB-1
capacidad	Número máximo de estudiantes	25
tipo	teoria, laboratorio o mixta	laboratorio

5.4. Sección PROFESOR

Registros de la forma `codigo;nombre;max_franjas;disponibilidad`.

Campo	Descripción	Ejemplo
<code>codigo</code>	Identificador único del profesor	P-001
<code>nombre</code>	Nombre completo	Ana Rojas
<code>max_franjas</code>	Máximo de franjas semanales asignables	12
<code>disponibilidad</code>	Rangos Df-Df separados por coma; campo vacío significa disponibilidad total	L1-L7,M1-M7,X1-X7

5.5. Sección CURSO

Registros de la forma `codigo;nombre;grupo;inscritos;semanales;duracion;requiere`.
Un mismo curso puede declararse varias veces, una por grupo.

Campo	Descripción	Ejemplo
<code>codigo</code>	Código del curso	IC-2001
<code>nombre</code>	Nombre del curso	Estructuras de Datos
<code>grupo</code>	Identificador del grupo	C1
<code>inscritos</code>	Número de estudiantes matriculados	30
<code>semanales</code>	Cantidad de sesiones por semana	2
<code>duracion</code>	Franjas consecutivas por sesión	2
<code>requiere</code>	teoria, laboratorio o mixta	mixta

El par (`codigo`, `grupo`) es la clave del grupo y debe ser único.

5.6. Sección IMPARTE

Registros de la forma `curso;grupo;profesor`. Indica qué profesor imparte un grupo. Un grupo debe tener al menos un profesor declarado.

5.7. Sección RESTRICCION

Cada registro comienza con el tipo de restricción seguido de sus argumentos. La aridad depende del tipo.

Tipo	Significado	Ejemplo
<code>aula_fija</code>	El grupo debe usar siempre esa aula.	<code>aula_fija;IC-2101;C1;LAB-1</code>
<code>prohibida</code>	Ninguna sesión del grupo puede cubrir la franja (día, franja) indicada.	<code>prohibida;MA-1002;C1;V5</code>
<code>excluyentes</code>	Dos grupos no pueden coincidir en el mismo día y franja, porque comparten estudiantes.	<code>excluyentes;IC-2001;C1;MA-1002;C1</code>
<code>prefiere</code>	Penalización w si el grupo no tiene una sesión que comience en esa franja (blanda).	<code>prefiere;IC-1802;C1;L1;3</code>
<code>compacta</code>	Penalización w por cada hueco del grupo (blanda).	<code>compacta;IC-1802;C1;5</code>

5.8. Ejemplo completo

```
# Ejemplo de instancia: Campus Central, periodo 2026-1

#:CONFIG
dias;L,M,X,J,V
franjas;7
inicio;07:00
duracion_franja;60

#:AULA
A-101;45;teoria
A-102;40;teoria
A-203;35;teoria
A-301;50;teoria
LAB-1;25;laboratorio
LAB-2;20;laboratorio

#:PROFESOR
P-001;Ana Rojas;12;L1-L7,M1-M7,X1-X7
P-002;Luis Mora;10;
P-003;Carmen Diaz;8;L1-L4,X1-X4
P-004;Jorge Solis;16;

#:CURSO
IC-1802;Introduccion a la Programacion;C1;35;2;2;teoria
IC-1802;Introduccion a la Programacion;C2;38;2;2;teoria
IC-2001;Estructuras de Datos;C1;30;2;2;mixta
IC-2101;Circuitos Digitales;C1;20;1;3;laboratorio
MA-1002;Calculo I;C1;40;3;1;teoria

#:IMPARTE
IC-1802;C1;P-001
IC-1802;C2;P-002
IC-2001;C1;P-003
IC-2101;C1;P-004
MA-1002;C1;P-001

#:RESTRICCION
aula_fija;IC-2101;C1;LAB-1
prohibida;MA-1002;C1;V5
prefiere;IC-1802;C1;L1;3
compacta;IC-1802;C1;5
```

5.9. Errores de formato

El programa deberá rechazar el archivo e informar el número de línea cuando detecte alguna de las siguientes situaciones:

- Aparece un registro antes de cualquier sección.
- Falta una clave obligatoria en **CONFIG**.
- Un campo numérico no es un entero o es negativo.
- Un código de día no declarado en **dias**.
- Una franja fuera del rango $1 \dots |F|$.

- Un tipo de aula o de requerimiento desconocido.
- Una referencia a un aula, profesor, curso o grupo inexistente.
- Un par (codigo, grupo) duplicado.
- Una franja de inicio tal que la sesión exceda el último índice de franja del día.

6. Formato de salida

El programa escribirá un archivo de texto plano en el nombre indicado por el usuario. El archivo consta de cinco bloques claramente delimitados.

6.1. Encabezado

```
=====
          ASIGNACION DE HORARIOS UNIVERSITARIOS
=====
Instancia : campus-central.dat
Estrategia : clpfd (first-fail)
Estado : SOLUCION ENCONTRADA
Fecha : 2026-01-15 10:42:07
-----
```

El campo Estado sólo admite uno de los siguientes valores: SOLUCION ENCONTRADA, SIN SOLUCION, LIMITE ALCANZADO o INSTANCIA INVALIDA.

6.2. Detalle de asignaciones

Una línea por sesión programada, ordenadas por curso, grupo y día.

```
DETALLE DE ASIGNACIONES
# Curso Grupo Profesor Aula Dia Franjas
1 IC-1802 C1 P-001 A-101 L 1-2
2 IC-1802 C1 P-001 A-101 J 1-2
3 IC-1802 C2 P-002 A-102 M 1-2
4 IC-1802 C2 P-002 A-102 X 1-2
5 IC-2001 C1 P-003 A-203 L 3-4
6 IC-2001 C1 P-003 A-203 X 3-4
7 IC-2101 C1 P-004 LAB-1 X 1-3
8 MA-1002 C1 P-001 A-301 L 3
9 MA-1002 C1 P-001 A-301 J 3
10 MA-1002 C1 P-001 A-301 X 4
```

6.3. Horario por grupo

Una rejilla por cada par curso-grupo, con una fila por franja y una columna por día. La celda contiene el curso y el grupo que ocupan esa franja, o - si está libre.

```
HORARIO POR GRUPO: IC-1802 C1
franja L M X J V
1 IC-1802/C1 -- -- IC-1802/C1 --
2 IC-1802/C1 -- -- IC-1802/C1 --
3 -- -- -- -- --
```

De forma análoga se emite el **horario por profesor** y, si el usuario lo solicita con la opción correspondiente, el **horario por aula**.

6.4. Resumen

```
RESUMEN
Sesiones programadas ..... 10
Franjas-sesion ocupadas ..... 18
Huecos de grupo ..... 0
Huecos de profesor ..... 0
Costo de restricciones blandas .. 0
Nodos explorados ..... 1482305
Retrocesos (backtracks) ..... 912744
Tiempo de CPU (s) ..... 3.41
Tiempo de pared (s) ..... 3.48
Nodos por segundo ..... 434694
Estado final ..... solucion valida de costo 0
```

Si la instancia no admite solución, el resumen se sustituye por un bloque **DIAGNOSTICO** que informará, como mínimo: el número de sesiones a programar frente al número de franjas disponibles, el grupo o el profesor que concentra la mayor demanda, y la restricción declarada que primero hizo imposible la búsqueda.

6.5. Salida legible por máquina

Cuando se invoque la opción correspondiente, el programa escribirá además un archivo Prolog con un hecho `asignacion/7` por cada sesión programada, con la forma:

```
asignacion(Curso, Grupo, Profesor, Aula, Dia, FranjaInicio, Duracion).

asignacion(ic_1802, c1, p_001, a_101, l, 1, 2).
asignacion(ic_1802, c1, p_001, a_101, j, 1, 2).
asignacion(ic_2101, c1, p_004, lab_1, x, 1, 3).
asignacion(ma_1002, c1, p_001, a_301, l, 3, 1).
```

Este archivo deberá poder consultarse directamente desde GNU Prolog y deberá ser idéntico, en contenido, al horario descrito en el archivo de texto.

7. Parte I: representación y validación en GNU Prolog

GNU Prolog será responsable de leer la instancia, validarla y construir la representación interna del problema.

La representación deberá basarse en hechos y reglas, por ejemplo:

```
dia(l). dia(m). dia(x). dia(j). dia(v).

aula(a_101, 45, teoria).
aula(lab_1, 25, laboratorio).

profesor(p_001, 'Ana Rojas', 12, [l1-l7, m1-m7, x1-x7]).

grupo(ic_1802, c1, 'Introduccion a la Programacion',
      35, 2, 2, teoria).
```

El programa deberá implementar, como mínimo, los siguientes predicados:

- `cargar_instancia(+Archivo)`: lee y valida el archivo.
- `valida/1`: comprueba la coherencia de la instancia y reporta todos los errores encontrados.
- `sesiones(-Lista)`: construye la lista de sesiones a programar.

- `dominio(+Sesion, -Dominio)`: calcula el dominio de una sesión aplicando ya las restricciones de capacidad, tipo y disponibilidad.
- `resolver(+Estrategia, -Horario)`: obtiene una solución.
- `verifica(+Horario)`: comprueba todas las restricciones duras.

Los predicados deberán ser declarativos en la medida de lo posible y no deberán depender de la forma en que se invoquen.

8. Parte II: resolución

8.1. Estrategia de backtracking

La primera estrategia, obligatoria, deberá construirse con *generación y prueba* sobre el motor de *backtracking* de Prolog. Cada sesión se instancia con un valor de su dominio y, una vez completa la asignación, se verifica el cumplimiento de las restricciones.

Se deberá implementar además una versión con **verificación anticipada**: cada vez que se instancia parcialmente una asignación, se comprueba inmediatamente aquello que ya puede decidirse (solapamiento de aula, de profesor y de grupo). Las asignaciones parciales inconsistentes deben descartarse antes de continuar la exploración.

8.2. Estrategia con restricciones sobre dominios finitos

La segunda estrategia deberá aprovechar el resolutor de dominios finitos incluido en GNU Prolog. Se espera el uso de al menos:

```
fd_domain(Variables, Min, Max)
fd_all_different(Lista)
fd_labeling(Variables, Opciones)
```

La modelación con dominios finitos exige reducir cada asignación a índices enteros. Por ejemplo, si se indexan las aulas y las franjas de forma consecutiva, una sesión puede representarse mediante tres variables enteras:

$$\text{aula} \in [0, |A| - 1], \quad \text{dia} \in [0, |D| - 1], \quad \text{franja} \in [1, |F| - d_c + 1]$$

El informe deberá explicar cómo se tradujeron las restricciones del problema a restricciones sobre estas variables y qué restricciones no pudieron expresarse de forma puramente extensional, junto con la estrategia utilizada para manejarlas.

8.3. Heurísticas

Deberán implementarse y compararse al menos las siguientes heurísticas de selección de variable y de valor:

- **Orden original**: las sesiones en el orden de lectura.
- **MRV** (*minimum remaining values*): primero la sesión con menor número de valores aún posibles.
- **Grado**: primero la sesión que aparece en más restricciones con otras.
- **Selección de valor menos restrictiva** (*least constraining value*): el valor que descarta menos opciones en las sesiones restantes.

- **Ordenamiento por demanda:** primero las sesiones de los cursos con mayor número de inscritos o de mayor duración.

Estas heurísticas pueden implementarse de forma explícita o activarse mediante las opciones de `fd_labeling/2` cuando correspondan (`ff`, `ffc`, `min`, `max`). La equivalencia deberá justificarse en el informe.

8.4. Reducción de simetría

Deberán aplicarse al menos dos mecanismos de reducción de simetría, entre ellos:

- Ordenar canónicamente las sesiones de un mismo grupo, de modo que la sesión i no pueda situarse antes que la sesión $i - 1$ en el orden cronológico.
- Fijar el orden de las sesiones equivalentes de grupos distintos que comparten profesor y características idénticas.
- Descartar, dentro de un mismo día, las permutaciones de sesiones de un grupo que ya hayan sido exploradas.

8.5. Optimización de restricciones blandas

Cuando se consideren las restricciones blandas, la búsqueda deberá encontrar la solución de costo mínimo mediante *branch and bound*: se explora el árbol y cada vez que se encuentra una solución de costo c se podan todas las ramas cuyo costo parcial sea mayor o igual que c .

El programa deberá informar el costo de la solución entregada y, cuando la búsqueda se haya completado sin alcanzar el límite, deberá indicar si el resultado es óptimo.

9. Interfaz mínima

El sistema deberá proporcionar un guion de envoltura que permita ejecutar al menos:

```
./horarios.sh campus-central.dat salida.txt bt
./horarios.sh campus-central.dat salida.txt bt --heuristica=mrsv
./horarios.sh campus-central.dat salida.txt clpfd --heuristica=ff
./horarios.sh campus-central.dat salida.txt clpfd --optimizar --limite=5000000
```

El guion deberá invocar a GNU Prolog de la siguiente manera:

```
#!/bin/sh
# horarios.sh
gprolog --consult-file horarios.pl \
    --entry-goal "main('$1','$2','$3')"
```

El programa deberá ejecutarse también de forma interactiva:

```
$ gprolog --consult-file horarios.pl
| ?- main('campus-central.dat', 'salida.txt', clpfd).
```

Como alternativa se permite compilar el programa a código nativo con `gplc`. La interfaz podrá modificarse siempre que conserve la funcionalidad solicitada y que el programa continúe ejecutándose en Linux con GNU Prolog.

10. Modos de ejecución

El sistema deberá permitir comparar al menos las siguientes configuraciones:

1. Ejecución interpretada con `gprolog`.
2. Ejecución compilada con `gplc`.
3. Estrategia de backtracking sin heurísticas.
4. Estrategia de backtracking con verificación anticipada y MRV.
5. Estrategia con dominios finitos, sin restricciones blandas.
6. Estrategia con dominios finitos y optimización con *branch and bound*.

Se recomienda realizar experimentos con instancias de tamaño creciente, con un número de sesiones igual a:

10, 20, 40, 80

cuando el hardware disponible lo permita. Para cada configuración deberán registrarse los tiempos de ejecución y el número de nodos explorados.

Deberá existir además un límite de búsqueda configurable (número máximo de nodos o de retrocesos). Si el límite se alcanza sin encontrar solución, el programa deberá terminar de forma controlada e informar `LIMITE ALCANZADO`, sin producir un horario parcial ni un archivo de salida incompleto.

11. Diagnóstico y tolerancia a errores

El sistema deberá ser capaz de detectar y reportar, sin abortar de forma abrupta, las siguientes situaciones:

1. El archivo de instancia no existe o no puede abrirse.
2. El archivo contiene errores de sintaxis o de coherencia.
3. La instancia es inconsistente por construcción: un grupo con más sesiones que franjas disponibles, un grupo cuyo número de inscritos excede la capacidad de todas las aulas compatibles, o un profesor sin disponibilidad suficiente para las sesiones que imparte.
4. La instancia no admite solución, aun siendo consistente.
5. La búsqueda alcanza el límite de nodos configurado.
6. El archivo de salida no puede escribirse.

En los casos 1, 2, 3 y 6 el programa deberá informar el error y terminar con un código de salida distinto de cero. En el caso 4 el programa deberá producir el bloque de diagnóstico descrito en la sección **Formato de salida**, identificando la restricción que restringe más el problema.

Como extensión, se recomienda implementar un mecanismo que, ante una instancia insatisfacible, determine un subconjunto mínimo de restricciones declaradas cuya eliminación haría la instancia resoluble.

12. Evaluación del rendimiento

El informe deberá analizar el comportamiento del sistema utilizando como mínimo las siguientes métricas.

12.1. Tiempo de ejecución

Para cada configuración e , se deberá medir:

$$T_e$$

correspondiente al tiempo de CPU necesario para resolver la instancia completa. Se medirá con `statistics/2` y se contrastará con el tiempo de pared obtenido con la orden `time` del sistema.

12.2. Número de nodos

Se deberá registrar el número de nodos explorados N_e y el número de retrocesos. El conteo se implementará con un contador explícito, ya sea mediante un acumulador en el predicado de búsqueda o mediante `assertz/1` y `retract/1`.

12.3. Tasa de exploración

Se deberá calcular:

$$R_e = \frac{N_e}{T_e}.$$

12.4. Aceleración

Se deberá calcular la aceleración obtenida por las heurísticas y por el ordenamiento, tomando como referencia la versión más ingenua:

$$S = \frac{T_{\text{ingenuo}}}{T_{\text{optimizado}}}.$$

12.5. Eficiencia de la poda

También deberá calcularse la proporción de nodos evitados:

$$E = \frac{N_{\text{ingenuo}} - N_{\text{optimizado}}}{N_{\text{ingenuo}}}.$$

El valor de E pertenece al intervalo $[0, 1]$; un valor cercano a 1 indica una poda muy efectiva.

Los resultados deberán presentarse mediante tablas y gráficos. El análisis deberá explicar las razones por las cuales la aceleración obtenida puede ser menor que la esperada. Entre los aspectos que deberán considerarse se encuentran:

- El costo de evaluar las heurísticas en cada nodo.
- El orden de las cláusulas y su efecto sobre las elecciones del motor de backtracking.
- El costo de la propagación de restricciones en dominios finitos.
- La copia y el recorrido de listas en cada paso de la búsqueda.
- El uso de `assertz/1` y `retract/1` durante la búsqueda.

- La lectura, el análisis y la validación del archivo de entrada.
- La estructura de la instancia: instancias muy restringidas admiten poca poda y las muy libres favorecen la búsqueda.
- La diferencia entre la ejecución interpretada y el código nativo generado con `gplc`.

13. Extensiones

Además del modelo obligatorio, se deberá implementar al menos **una extensión**. Algunas posibilidades son:

- Optimización con *branch and bound* sobre restricciones blandas.
- Salida en formato CSV además del formato de texto.
- Generador aleatorio de instancias, parametrizado por número de cursos, aulas y franjas, que garantice instancias resolubles.
- Soporte para franjas de distinta duración o para días no consecutivos.
- Soporte para más de un profesor por grupo, con reparto de sesiones.
- Restricción de franjas de almuerzo comunes a toda la universidad.

Como extensión adicional, se podrá permitir que el usuario especifique una secuencia de criterios de optimización aplicados en orden de prioridad, por ejemplo:

```
criterios([prefiere, compacta, huecos_profesor, huecos_grupo]).
```

En ese caso, el programa deberá resolver de forma lexicográfica: minimizar el costo del primer criterio, y sólo entre las soluciones óptimas de ese criterio minimizar el siguiente.

14. Entregables

El proyecto deberá incluir:

1. Instancias de prueba

Un conjunto de al menos cinco instancias utilizadas para verificar el funcionamiento del sistema, incluyendo una instancia pequeña, una grande, una sin solución y una con errores de formato.

2. Manual de ejecución

Instrucciones suficientes para instalar GNU Prolog, cargar el programa, ejecutar el guion de envoltura, interpretar los archivos de entrada y de salida, y reproducir los experimentos.

3. Informe técnico

El informe deberá explicar:

- Modelo del problema y representación interna.
- Estrategias de resolución implementadas.
- Definición de las restricciones duras y blandas.
- Heurísticas y mecanismos de poda utilizados.
- Reducción de simetría aplicada.

- Lectura, validación y escritura de archivos.
- Manejo de instancias inválidas o insatisfacibles.
- Resultados experimentales.
- Análisis de la aceleración y de la eficiencia de la poda.
- Limitaciones del sistema.

4. Demostración

El proyecto deberá ser presentado y ejecutado durante la demostración correspondiente.

15. Restricciones

- Todo el programa deberá estar implementado en GNU Prolog y ejecutarse en Linux. Los guiones de *shell* sólo podrán utilizarse como envoltorio de invocación.
- Las estrategias de backtracking puro y con dominios finitos son obligatorias y deberán coexistir en el mismo programa.
- No se permite utilizar ningún resolutor de restricciones ni biblioteca externa de planificación, más allá del módulo de dominios finitos incluido en GNU Prolog.
- No se permite incorporar la solución esperada de ninguna instancia dentro del programa.
- No se permite leer la instancia con utilidades externas al programa: la lectura y el análisis del archivo forman parte del proyecto.
- El programa no deberá requerir conexión a red ni acceso a servicios externos.
- El programa deberá ser ejecutable con las versiones de GNU Prolog indicadas por el profesor y no podrá depender de extensiones propias de otros sistemas Prolog.
- La interfaz podrá modificarse libremente siempre que se conserven las funcionalidades descritas en las secciones **Interfaz mínima** y **Modos de ejecución**.

16. Criterios de evaluación

La calificación se distribuirá de la siguiente manera:

Componente	Porcentaje
Representación del problema en GNU Prolog	20 %
Implementación de la búsqueda y de las restricciones	25 %
Lectura, validación y escritura de archivos	20 %
Correctitud del horario respecto de las restricciones duras	15 %
Optimización de las restricciones blandas	10 %
Informe y análisis experimental	10 %
Total	100 %

17. Defensa del proyecto

Durante la defensa, los estudiantes deberán ser capaces de explicar cualquier parte del código desarrollado.

El evaluador podrá solicitar modificaciones pequeñas al programa durante la defensa, por ejemplo:

- Cambiar el número de días o de franjas horarias.
- Agregar o eliminar aulas.
- Incorporar una nueva restricción dura.
- Incorporar una nueva restricción blanda con su peso.
- Cambiar la heurística de selección de variable o de valor.
- Proponer una instancia sin solución y analizar el diagnóstico.
- Aumentar el tamaño de la instancia y comparar los tiempos.
- Modificar el formato de la salida.
- Aislar y explicar una violación de restricción provocada deliberadamente.

La defensa deberá demostrar que los integrantes comprenden tanto el modelo lógico del problema como las estrategias de búsqueda implementadas.

La fecha de entrega será el día 27 de noviembre.